

Software Supply Chain Attacks

Build Provenance

NG, Chun Sing (r14922186)

OU, Chia-Yun (r14922072)

Our Daily Routine

Every day, developers run:

```
npm install  pip install  mvn install  conda install
```

These download and execute **third-party code** — one dependency can pull in **dozens of transitive packages** — without inspection.

Do we ever verify they haven't been tampered with?

Background

Supply chain attacks target the *distribution pipeline*, not the application code.

Documented across every major ecosystem:

- **npm** — event-stream (2018), axios (2026), TanStack (2026)
- **PyPI** — typosquatting and dependency confusion campaigns
- **Maven, RubyGems, cargo** — documented backdoor attempts

Case Study: axios — March 31, 2026

100M weekly downloads · stolen maintainer credentials · 3-hour exposure window

Attacker published a malicious version that injected a hidden dependency:

```
npm install axios
├─ malicious axios version
│   └─ attacker-controlled package
│       └─ runs postinstall script ← Remote Access Trojan (RAT) installed silently
```

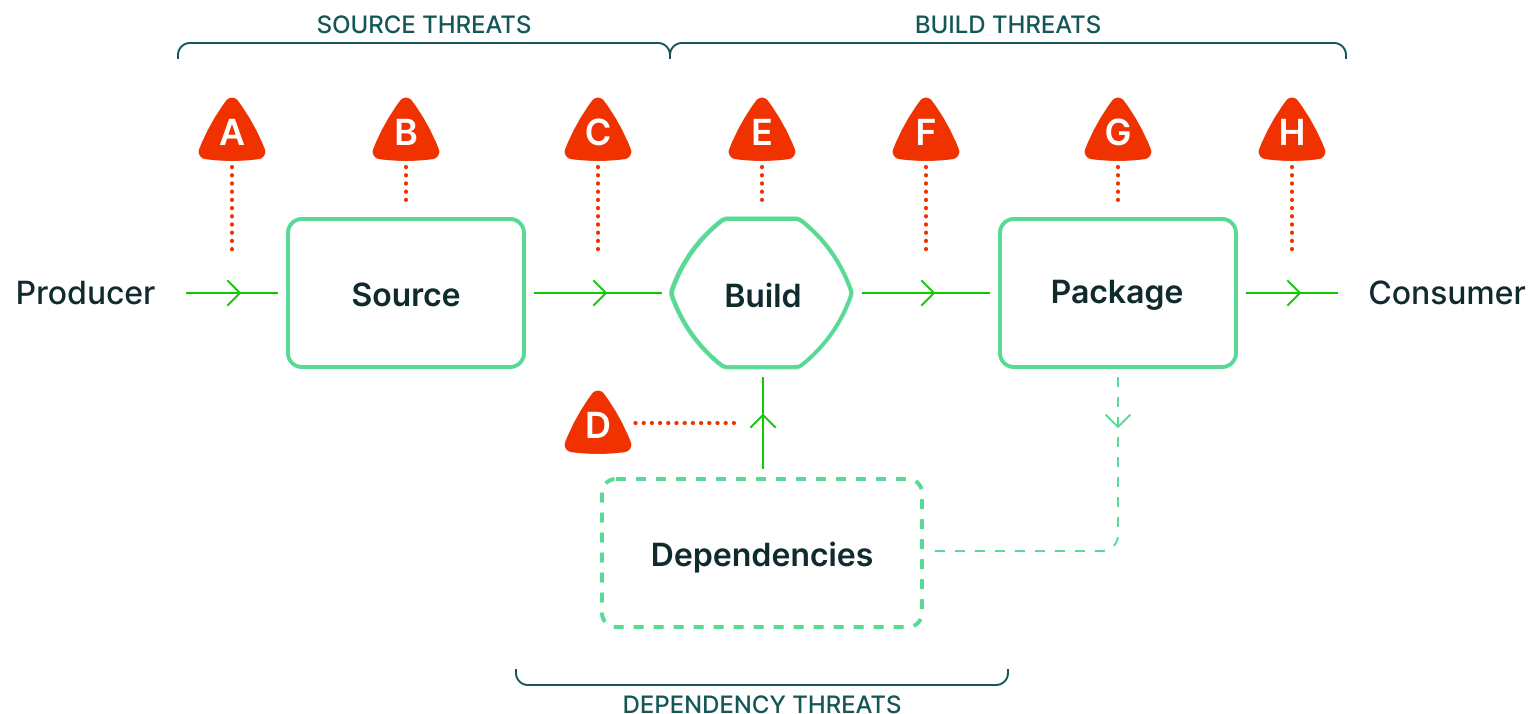
No prompt. No review. Compromised upon installation.

OpenAI's macOS app-signing pipeline was affected — putting code-signing certificates for **ChatGPT Desktop** and **Codex** at risk.

Common Defenses

Tool	What it protects	Limitation
<code>npm audit</code> / Snyk	Known CVEs in the dependency tree	Misses new malicious versions — no CVE assigned yet
<code>npm ci</code> + lock file	Hash-verifies pinned versions against the registry	Trusts the lock file — which was generated by <code>npm install</code>
Dependabot	Auto-updates vulnerable deps	Still CVE-dependent; an upgrade could introduce a malicious version

The Supply Chain Has Many Attack Surfaces



SOURCE THREATS

- A** Submit unauthorized change
- B** Compromise source repo
- C** Build from modified source

DEPENDENCY THREATS

- D** Use compromised dependency

BUILD THREATS

- E** Compromise build process
- F** Upload modified package
- G** Compromise package registry
- H** Use compromised package

The Core Gap: No Tamper-Evident Chain

Between source and install, no one can verify:

- Built from the **correct source commit**?
- **Build steps** unmodified?
- **Published tarball** matches CI output?

Attacks are **silent**.

Build Provenance — A Solution

Proposed in 2021: SLSA framework (Google) · Sigstore (OpenSSF)

"Built from commit X · by pipeline Y · at time T" — signed and logged.

Question	Answer
Correct source commit?	Commit hash recorded and signed
Build steps unmodified?	Pipeline identity bound via OIDC (OpenID Connect)
Tarball matches CI output?	Artifact hash logged in Rekor (public append-only log)

Verification — Turning the Record into Prevention

Provenance is a record. Prevention requires **verification** at install time.

SLSA defines three steps (consumer's responsibility):

1. Verify the provenance **signature**
2. Compare against **expected** source repo, builder, and build type
3. *(Optional)* Recursively verify dependencies

Expected values are set by the maintainer, declared in the repo, or learned from the first observed attestation.

Technology Stack

Layer	Technology	Role
Framework	SLSA	Defines <i>what</i> provenance must prove (Build Levels 0–3)
Cryptography	Sigstore	Keyless signing; public transparency log (Rekor)
Platform	Registry + CI	Generates and stores attestations (npm, PyPI, ...)
Consumer	npq · Socket Firewall	Pre-install gate: provenance verification

Workflow

Producer — at publish time:

1. GitHub Actions builds from source → Sigstore signs and logs the attestation (commit · builder · artifact hash)
2. Registry stores `dist.attestations` alongside the package

Consumer — at install time:

3. `npm install <package>` fetches the package and its attestations
4. Decision:
 - Prior had no attestation → **WARNING**
 - Prior had attestation, target **does not** → **ERROR**
 - Target has attestation → verify signature → install proceeds

Example Usages of npq

Prior had no attestation

```
(base) peter@DESKTOP-D6Q4KQE:~/security-2026$ docker run -it --rm supply-chain-demo bash
root@a14bfb9bbb93:/demo# npq install event-stream

Packages with issues found:

> event-stream@latest

✗ Package Health      · Package repository has been archived on GitHub
⚠ Package Health      · Detected an old package: created 7 years ago
⚠ Supply Chain Security · Unable to verify provenance: the package was published without any attestations
```

Prior had attestation

```
(base) peter@DESKTOP-D6Q4KQE:~/security-2026$ docker run -it --rm supply-chain-demo bash
root@b95fd2003b42:/demo# npq install pino@9.14.0

Packages with issues found:

> pino@9.14.0

✗ Supply Chain Security · Provenance regression detected: published version 9.13.1 includes npm provenance metadata, but pino@9.14.0 does not (or it could not be verified).
```

Adoption

Provenance is available — uptake is still early.

Ecosystem	Status	Adoption	Source
npm	GA since Oct 2023	3,800 packages in beta (as of Oct 2023)	Sigstore 2023
PyPI	GA since Nov 2024	17% of all uploads (end 2025)	PyPI 2025
GitHub Attestations	GA since Jun 2024	—	GitHub 2024
Maven	Opt-in since Jan 2025	Early adopters only	Sonatype 2025
RubyGems	In progress	20 top gems (Mar 2025)	RubyGems 2025
Conda	Beta since Aug 2025	—	prefix.dev 2025

npq: 1.6k GitHub stars · Main barrier: maintainer awareness and workflow migration overhead.

(sources: Appendix — GA dates · adoption figures)

The Limitation: TanStack — May 11, 2026

- 42 packages · 84 malicious versions published in minutes
- Payload harvested cloud credentials, API tokens, and SSH keys
- Detected in 26 minutes by an external researcher
- Published with valid build provenance

How the Attack Worked

A fork PR poisoned the shared CI cache.

When a legitimate release ran, it restored the cache — which extracted the OIDC token from runner memory and pushed malicious packages.

"Our own CI pipeline stole its own publish token — by way of a cache that everyone in the chain implicitly trusted."
— TanStack postmortem

The Limit of Provenance

Provenance correctly attested the package came from TanStack's workflow. However, the workflow had been compromised.

"Provenance shouldn't be confused with innocence."

Build provenance is **necessary but not sufficient**.

Industry Response: OpenAI

Two OpenAI employee devices were affected during the TanStack incident — during a **phased rollout** of controls that would have prevented it.

Controls already deployed after the axios incident:

- Hardening of credential materials in CI/CD pipelines
- Package manager config: `minimumReleaseAge` (rejects newly published versions)
- Security software to **validate provenance** of new packages

Takeaways

Build provenance raises the bar — but the bar can still be cleared.

- **Adopt** provenance: supported natively by npm, PyPI, and GitHub Actions
- **Verify** at install time: npq, Socket Firewall
- **Harden beyond** provenance: isolate CI pipelines, restrict cache access, audit workflow permissions

References

- SLSA — *Supply chain threats*, slsa.dev/spec/v1.0/threats-overview
- SLSA — *About SLSA*, slsa.dev/spec/v1.2/about
- SLSA — *Verifying artifacts*, slsa.dev/spec/v1.0/verifying-artifacts
- Socket.dev — *Introducing Socket Firewall*, socket.dev/blog/introducing-socket-firewall
- npq repository — github.com/lirantal/npq

References

- npm Blog — *Details about the event-stream incident*, 2018
blog.npmjs.org/post/180565383195/details-about-the-event-stream-incident
- GAO Blog — *SolarWinds Cyberattack Demands Significant Federal and Private Sector Response*, 2021
gao.gov/blog/solarwinds-cyberattack-demands-significant-federal-and-private-sector-response-infographic
- Microsoft Security Blog — *Mitigating the Axios npm supply chain compromise*, Apr. 2026
microsoft.com/en-us/security/blog/2026/04/01/mitigating-the-axios-npm-supply-chain-compromise
- Axios — *Post Mortem: axios npm supply chain compromise*, github.com/axios/axios/issues/10636
- OpenAI — *Our response to the Axios developer tool compromise*, Apr. 2026
openai.com/index/axios-developer-tool-compromise
- TanStack — *Postmortem: TanStack npm supply-chain compromise*, tanstack.com/blog/npm-supply-chain-compromise-postmortem
- TanStack — *Hardening TanStack After the npm Compromise*, tanstack.com/blog/incident-followup
- OpenAI — *Our response to the TanStack npm supply chain attack*, 2026
openai.com/index/our-response-to-the-tanstack-npm-supply-chain-attack

Appendix: GA Dates

Ecosystem	Event	Date	Source
npm	<code>npm publish --provenance</code> GA	Oct 2023	github.blog/security/supply-chain-security/introducing-npm-package-provenance
GitHub Attestations	Artifact Attestations GA	Jun 2024	github.blog/changelog/2024-06-25-artifact-attestations-is-generally-available
PyPI	Sigstore attestations GA	Nov 2024	blog.pypi.org/posts/2024-11-14-pypi-now-supports-digital-attestations
Maven	Sigstore opt-in	Jan 2025	central.sonatype.org/news/20250128_sigstore_signature_validation_via_portal
Conda	Sigstore beta	Aug 2025	prefix.dev/blog/securing-the-conda-package-supply-chain-with-sigstore

Appendix: Adoption Figures

Ecosystem	Statistic	Source
npm	3,800 packages adopted during beta; 500M+ downloads of provenance-enabled packages	Sigstore Blog, <i>npm's Sigstore-powered provenance goes GA</i> — blog.sigstore.dev/npm-provenance-ga
PyPI	17% of all uploads included attestations (end 2025)	PyPI, <i>2025 Year in Review</i> — blog.pypi.org/posts/2025-12-31-pypi-2025-in-review
RubyGems	20 top gems shipping attestations (Mar 2025)	RubyGems Blog, <i>February 2025 Updates</i> — blog.rubygems.org/2025/03/19/february-rubygems-updates.html
GitHub / Maven / Conda	No public adoption figures	—